Software engineering practices and AI tools for research code

William S Hart

Infectious Disease Modelling Group
Mathematical Institute, University of Oxford

Group meeting, 29 October 2026



Oxford Mathematics

University of Oxford
Mathematical Institute

This talk

- Our results are only as reliable as the code behind them.
- **Part 1:** practices and tools for reliable, reproducible code.
- **Part 2:** AI coding tools: uses, good habits and risks.

■■■■■ Importance for a typical IDM project (5 = essential for every project)

my choice alternatives Tools for Python, R and Julia

⚠ **Disclaimer:** these slides were generated by AI from my input. Evaluate any advice independently before following it.

Nine practices, ranked by importance



Version control





Environments





Testing





Archiving code





Formatting and linting





Workflow management





Editor and debugger





Type checking





Continuous integration



Version control

Every change recorded, every mistake undoable.

Importance ■■■■■

- See what changed, when and why.
- Backup and collaboration in one.
- Branches let features develop in parallel.
- Commit before an agent edits, then review its diff.

Tools

Any	git
Hosting	GitHub, GitLab
Interfaces	VS Code, GitHub Desktop, GitHub CLI (gh)

Environments and package management

Mathematical Institute

One environment per project, pinned by a lockfile. Importance ■■■■■

- Code depends on package versions (e.g. numpy 2).
- A shared global environment drifts and breaks old projects.
- A lockfile (exact versions of every package) lets anyone, and the cluster, rebuild it.

Tools

Python	**pixi** · uv · conda/mamba · pip + venv
R	**renv** · pak · rig
Julia	Pkg
Containers	Docker · Apptainer

Testing

How you know the code, yours or the AI's, is right.

Importance ■■■■□

- Test against known answers: analytic results, limiting cases, simulate-and-recover.
- Stochastic code: pass a seeded `rng` in; compare with tolerances.
- Write the failing test first (TDD): ideal with agents.
- Then refactor knowing nothing broke.

Tools

Python	**pytest** hypothesis pytest-cov
R	**testthat**
Julia	Test

Archiving code

A permanent copy of the code behind each paper.

Importance

- Archive the exact version used for the paper.
- A DOI keeps the paper's link permanent, even if the repo moves.
- Zenodo's GitHub integration needs a public repo; otherwise upload the release by hand.

Tools

Archives: **Zenodo** | Software Heritage | OSF

Formatting and linting

University of Oxford
Mathematical Institute

Consistent, bug-checked code for free.

Importance ■■■□□

- Consistent style, with no effort or arguments.
- Linters catch real bugs, e.g. undefined names, unused imports.
- Runs on save, or after each agent edit (a hook: a command run automatically).

Tools

Python	**ruff** pylint
R	air styler lintr
Julia	JuliaFormatter.jl Runic.jl
Any	pre-commit

Workflow management

University of Oxford
Mathematical Institute

One command rebuilds every result.

Importance ■■■□□

- Reruns only what changed when code or data change.
- Makes the dependencies between data, code and outputs explicit.
- Scales to the cluster; matters more as projects grow.

Tools

Any	**Snakemake** Nextflow Make
R	targets
Simple	pixi tasks

Editor and debugger

See errors as you type; inspect, don't print.

Importance ■■■□□

- The editor flags errors and makes code easy to navigate.
- A debugger pauses the code so you can inspect its state.
- Notebooks for exploring; scripts once it matters.

Tools

Editors	**VS Code**	PyCharm	Positron	RStudio
Debuggers	**VS Code debugger**	pdb	R browser()	Infiltrator.jl

Type checking

University of Oxford
Mathematical Institute

Catch wrong-type bugs before running.

Importance ■■□□□

- E.g. passing `None` where an array is expected.
- Type hints document interfaces, and help editors and AI.
- Lower priority for exploratory code, but cheap to add with AI.

Tools

Python	**ty (beta)** · pyrefly · mypy · pyright
Julia	JET.jl
R	none mainstream

Continuous integration

University of Oxford
Mathematical Institute

Tests run on every push, on clean machines.

Importance ■■□□□

- Catches “works on my machine” and broken lockfiles.
- Tests several operating systems at once.
- Essential for packages; optional for analysis repos.

Tools

Pipelines	**GitHub Actions**	GitLab CI
Coverage	**Codecov**	

Packages other people install: ■■■■□

What goes in a repo

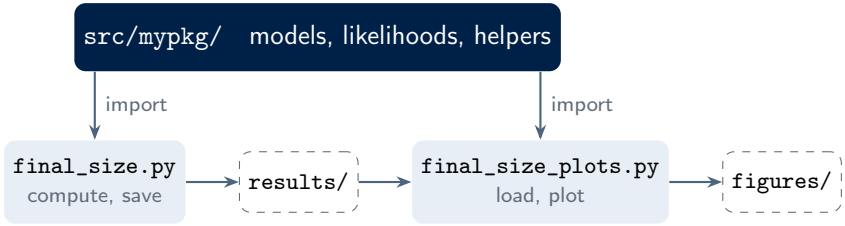
```
project/
├── README.md   what it is, install, reproduce
├── LICENSE   no licence = no reuse
├── .gitignore
├── pyproject.toml   env spec (format: my choice)
├── pixi.lock   exact versions
├── AGENTS.md   + CLAUDE.md importing it
├── src/mypkg/   reusable code as a package
├── scripts/   analysis + plot script pairs
│   ├── final_size.py   compute, save results
│   └── final_size_plots.py   load results, plot
├── tests/
├── data/   inputs; never identifiable data or secrets
├── results/   outputs (committed: my choice)
├── figures/
└── Snakefile
```

Essential Good practice My choice

My choices → alternatives

- pixi → uv, conda `environment.yml`
- src layout (package under `src/`) → flat package
- Script pairs → notebooks
- Committed outputs → regenerate only
- Snakefile → pixi tasks, Make
- `CLAUDE.md` importing `AGENTS.md` → `AGENTS.md` or `CLAUDE.md` only

Organising code



- **Don't repeat yourself:** define each value once, e.g. the serial interval. A repeated one-liner is fine.
- **One function per task:** `simulate_outbreak`, `plot_risk`. Pass seeds in, no hidden globals.
- **Public vs private:** public functions are documented and tested; `_helpers` can change freely.

Documenting code

- README: what it is, how to install it, how to reproduce the results.
- Docstrings for public functions: purpose, arguments, returns.
- Comments say *why*, not *what*.
- Names say what things are, with units (`incubation_days`).
- No commented-out code: git remembers it.
- Doc websites only for shared packages: **Sphinx** | roxygen2 + pkgdown | Documenter.jl

Live demo: this repo

- Layout, README and licence.
- `pyproject.toml` and `pixi.lock`.
- `pixi run format`, `lint`, `typecheck`, `test`.
- The Snakefile.
- `AGENTS.md` and the hooks.

Further reading

- *Best practices for scientific computing*:
  Wilson et al. (2014), *PLoS Biol.*, https://doi.org/10.1371/journal.pbio.1001745
- *Good enough practices in scientific computing*:
  Wilson et al. (2017), *PLoS Comput. Biol.*, https://doi.org/10.1371/journal.pcbi.1005510
- *The Turing Way*, a handbook for reproducible research:
  `https://book.the-turing-way.org`
- Oxford Research Software Engineering group training:
  `https://www.rse.ox.ac.uk/training`

1. Practices and tooling

2. AI tools

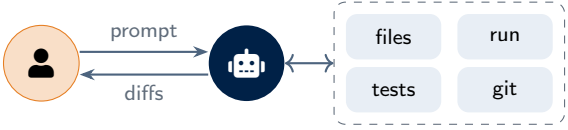
Options: chat assistants and coding agents

Chat
ChatGPT, claude.ai



You are the loop.
Good for questions, snippets.

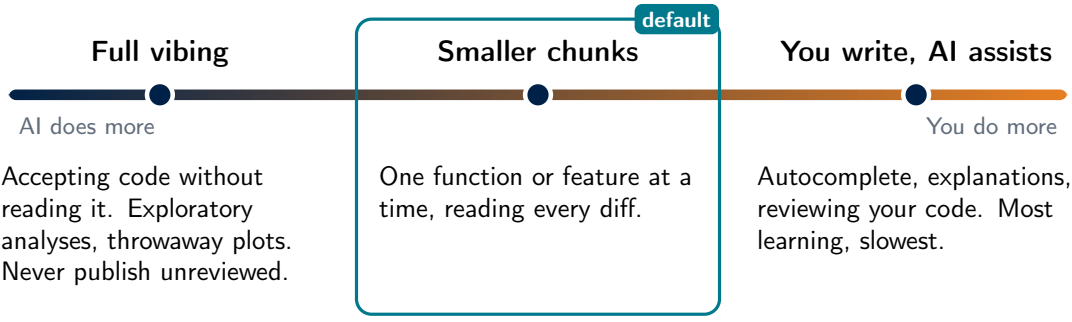
Coding agent
Claude Code,
Codex



It runs the loop in your repo; you review.

Claude Code terminal, VS Code, desktop, web OpenAI Codex CLI, IDE, cloud
Also: GitHub Copilot (agent mode), Cursor, Gemini CLI.

How much to delegate



Choose by how easily *you* can check the result.

Uses: checking and testing

- **Bugs:** ask it to review code, or give it the traceback; ask for the root cause, not a patch.
- **Smoketests:** "run a small case and check the output". Good checks become tests.
- **Writing tests:** good at boilerplate and edge cases; you decide what's checked. Pairs well with TDD.
- **Watch for** tests that mirror the code, or get edited or skipped to pass.

Uses: grunt work and more

Grunt work

Tedious for you, quick for you to check.

- Type hints and docstrings
- Renames across files
- New-but-similar sensitivity analyses
- Refactoring duplicated code
- Updating for new library versions

And more

Reading, translating, explaining.

- Understanding unfamiliar code
- Translating R or MATLAB to Python
- Decoding install and HPC errors
- READMEs, commits, merge conflicts
- Slides for a supervisor meeting

Working with an agent: prompting

- **Be specific:** the files involved, a pattern to follow, what "fixed" looks like.
- **Let it check its work:** tests or a known result. Ask for evidence.
- **Bigger tasks:** plan first, or ask it to interview you.
- **Exploring:** vague is fine ("what would you improve here?").

Vague

"fix the bug in the plot"

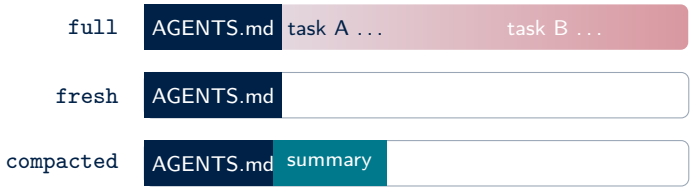
Specific

"the risk curve in `figures/final_size/` starts at day 1, not day 0. Check how the time axis is built in `scripts/final_size_plots.py`, fix it, and show me the corrected figure"

Docs: code.claude.com/docs/en/best-practices

Working with an agent: sessions and context

University of Oxford
Mathematical Institute



- Two failed corrections? Start afresh with a better prompt.
- Interrupt early. Checkpoints undo its file edits, but not commands it ran: git covers those.
- Review in a fresh session or with a second model, unbiased by having written it.

Docs: code.claude.com/docs/en/best-practices

Plan the science before the code

You decide, before any code

- The model and its assumptions.
- Parameterisation and units (gamma: scale or rate?).
- What correct output looks like: known answers and limiting cases, which become your tests.

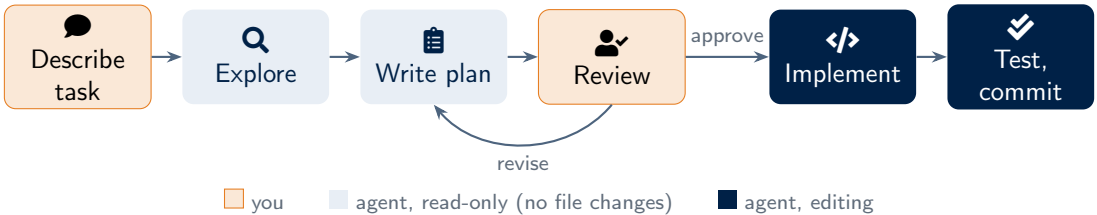
Talk it through with the agent if you like; the decisions are yours.

AI can help with the maths

- Deriving results, e.g. the final size or R_0 of your model.
- Checking your derivations for slips.
- Finding limiting cases and known answers to test against.
- Checking the code matches the equations.

✔ Verify independently: by hand, with SymPy, or numerically.

Plan first: agree the plan before any file changes



✔ Use it

When you're unsure of the approach, or several files change: use plan mode, or ask for a plan and no edits.

✖ Skip it

When you could describe the change in one sentence.

Docs: code.claude.com/docs/en/permission-modes

Agent instruction files

Mathematical Institute

Project
Committed (`AGENTS.md`): the project's conventions, for everyone.

Personal, this project
Your preferences; not committed.

Personal, all projects
In your home config.

- Loaded every session: commands and conventions the agent can't infer.
- Keep it short: would removing this line cause mistakes?
- Add a rule when you find yourself repeating a correction.
- `AGENTS.md` is an open format read by many coding agents.

Docs: code.claude.com/docs/en/memory

Skills

- Instructions plus templates and scripts, loaded when relevant or on request.
- Loaded only when used, so they can be long.
- **Make one** "when you keep pasting the same instructions, checklist, or multi-step procedure into chat" (within or across projects).
- Examples: `research-repo-skeleton` set up this repo; `oxford-beamer` set up this deck.
- An open standard (agentskills.io), usable by other tools.

Docs: code.claude.com/docs/en/skills

Other useful features

Feature	Use
Hooks	Commands run automatically at set points, e.g. format and lint after every edit (as in this repo).
Checkpoints	Rewind the agent's file edits and the conversation.
Permission modes	Choose what it may do without asking, and know which mode you're in.
Sandbox	Limits the files and network its commands can reach.
Fresh-context review	A second session or model reviews the diff, unbiased by having written it.
Subagents	Helper agents with their own fresh context, for side investigations.
Parallel sessions	Separate checkouts (git worktrees), so several sessions can work at once.
Connectors (MCP)	External tools and data, e.g. PubMed, GitHub.

Names link to the Claude Code docs; most coding agents have equivalents.

Live demo: Claude Code

- Plan mode on a small task, following `AGENTS.md`.
- Hooks format and lint each edit.
- Review the diff, then undo with a checkpoint.

Risks: correctness and skills

Plausible but wrong

Runs and looks fine, but e.g. gamma scale vs rate, or an off-by-one in a renewal-equation convolution.

✔ Check against analytic results, limiting cases, simulate-and-recover.

Tests that pass for the wrong reason

Tests that mirror the code, or get edited or skipped to pass.

✔ Decide what the tests check, and read them.

Bloat and hidden fallbacks

More code than needed; fallbacks that hide errors.

✔ Small steps, read every diff, ask for simpler code.

Losing understanding

You can't defend code you don't understand.

✔ Ask it to explain; write some code yourself.

Risks: data, security and policy

Mathematical Institute

Confidential or patient-level data

Everything the agent reads or prints goes to the provider.

✔ Use University-approved tools (ChatGPT Edu, Codex; Claude under discussion); check data agreements; develop on synthetic data.

Destructive commands

Depending on the mode, agents run commands without asking.

✔ An ask-first mode or a sandbox for sensitive work; commit first.

Invented packages, hidden instructions

Packages that don't exist (or are malicious); instructions hidden in untrusted code or web pages.

✔ Check new dependencies in the lockfile diff; take care with unfamiliar repos.

Journal, funder and thesis rules

AI can't be an author; policies vary.

✔ Check them, and say how AI was used.

You own every line, whoever wrote it

- If you can't explain it, don't rely on it.
- Read every diff; ask about anything unclear.
- Plan first, then small steps, committing after each.
- Check against known answers, not the agent's say-so.
- Get a fresh-context review (a new session or a second model).
- Put corrections in `AGENTS.md` so they stick.

Delegate only what you can check

1 Plan



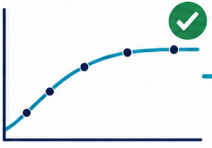
Plan before code
Model, parameters and units
Decide the checks first

2 Implement



Small, reviewable changes
Clear context for the agent
Read every diff

3 Validate



Known answers and limiting cases
Simulate and recover
Seeded RNGs and tolerances

4 Reproduce



One command: data to figures
Record inputs and parameters
Archive the exact version

FOUNDATIONS



Version control



Locked environment



Tests and automated checks



README + agent instructions

You own every line, whoever wrote it. Protect sensitive data.